LAPORAN
WORKSHOP SISTEM INFORMASI WEB SERVER
(TIF330805)
SEMESTER III



TUGAS PRAKTIKUM ACARA 24

NAMA:

RAJATA PRINGGIT BINTANG BIHAR

GOLONGAN:

A

NIM:

E41250264

PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)

TEKNIK INFORMATIKA KAB. NGANJUK

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI JEMBER

TAHUN 2026

PENDAHULUAN

1. Latar Belakang

Pada Acara 17 sampai 23, API Sistem Informasi Akademik sudah memiliki autentikasi, dokumentasi, dan hasil pengujian, tetapi masih diakses lewat Postman. Acara 24 membangun lapisan View pertama: sebuah halaman HTML yang mengambil data mahasiswa dari API dengan Fetch API lalu menampilkannya dalam tabel. Agar halaman pada origin yang berbeda dapat memanggil API, ditambahkan pula header CORS pada Front Controller.

2. Tujuan

1. Mengaktifkan header CORS pada public/index.php sehingga berlaku untuk seluruh resource.
2. Membuat folder public/views berisi index.html dan app.js.
3. Menulis kode fetch dengan async/await dan try-catch untuk menampilkan data mahasiswa.
4. Menguji tampilan dan memastikan tidak ada error CORS pada Developer Tools.
5. Menambahkan indikator "Memuat data..." (tugas mandiri 1).
6. Menambahkan penanganan data kosong dengan pesan "Belum ada data" (tugas mandiri 2).

3. Dasar Teori

Fetch API menyediakan antarmuka JavaScript modern untuk melakukan permintaan HTTP dan menggantikan XMLHttpRequest yang berbasis callback. Pemanggilan fetch() mengembalikan Promise yang selesai begitu server mengirim header response, sedangkan pembacaan body (misalnya lewat .json()) dilakukan pada tahap terpisah. Sintaks async/await membuat kode asynchronous dapat ditulis berurutan, dan kegagalan permintaan seperti jaringan terputus ditangani dengan blok try-catch di sekeliling await fetch(...).

Cross-Origin Resource Sharing (CORS) adalah mekanisme keamanan berbasis header HTTP yang menentukan apakah halaman web pada satu origin (skema, host, dan port) boleh mengakses resource dari origin lain. Browser menolak response dari origin lain kecuali server secara eksplisit mengizinkannya melalui header seperti Access-Control-Allow-Origin (Tabel 1).

Tabel 1. Header CORS yang digunakan

Header	**Fungsi**
Access-Control-Allow-Origin	Origin yang diizinkan mengakses resource (* berarti semua origin).
Access-Control-Allow-Methods	Method HTTP yang diizinkan pada permintaan lintas origin.
Access-Control-Allow-Headers	Header request yang boleh dikirim, misalnya Content-Type dan Authorization.

HASIL PRAKTIK

1. Implementasi

a. Perubahan File

Acara 24 mengubah Front Controller dan menambahkan dua berkas View. Dokumentasi API ikut diperbarui dengan keterangan CORS.

Tabel 2. Daftar perubahan file

File	Status	Keterangan
public/index.php	Diubah	Header CORS dan jawaban 204 untuk preflight OPTIONS
public/views/index.html	Baru	Halaman tabel data mahasiswa
public/views/app.js	Baru	Kode fetch, indikator memuat, data kosong, dan penanganan galat
docs/API_DOCUMENTATION.md	Diubah	Ditambah bagian 3.5 CORS
docs/openapi.yaml	Diubah	Keterangan CORS dan alamat server
README.md	Diubah	Ringkasan perubahan Acara 24

b. Langkah 1 - Mengaktifkan Header CORS pada Front Controller

Header CORS ditulis satu kali pada awal public/index.php, sebelum Router dijalankan, sehingga berlaku untuk seluruh resource (mahasiswa, login, register, dan resource baru pada acara berikutnya), termasuk response kesalahan seperti 404 dan 401. Selain tiga header dari modul, ditambahkan jawaban 204 untuk permintaan OPTIONS. Browser mengirim OPTIONS (preflight) sebelum permintaan POST, PUT, atau DELETE yang memakai header Authorization; tanpa jawaban ini Router akan menjawab 405 dan permintaan diblokir. Untuk GET sederhana pada acara ini preflight belum diperlukan.

```
<?php
// CORS: diizinkan untuk seluruh resource yang melewati Router (Acara 24)
header('Access-Control-Allow-Origin: *');
header('Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS');
header('Access-Control-Allow-Headers: Content-Type, Authorization');

// Preflight: browser mengirim OPTIONS sebelum POST/PUT/DELETE berheader Authorization.
// Dijawab 204 tanpa body dan tidak diteruskan ke Router.
if (($_SERVER['REQUEST_METHOD'] ?? '') === 'OPTIONS') {
    http_response_code(204);
    exit;
}

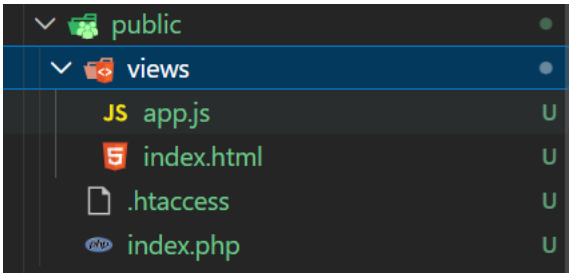
require_once __DIR__ . '/../app/core/autoload.php';
(new Router())->dispatch();
```

c. Langkah 2 - Membuat Folder View

Folder views dibuat di dalam public/ karena berkas HTML dan JavaScript harus dapat dibuka langsung oleh browser sebagai berkas statis. Berkas yang ada di public/ dan bukan folder

diteruskan apa adanya oleh aturan RewriteCond pada .htaccess, sehingga tidak melewati Router.

```
public/
├── views/
│   ├── app.js
│   └── index.html
├── .htaccess
└── index.php
```



Gambar 1. Struktur folder public/views

d. Langkah 3 - Menulis Kode Fetch untuk Menampilkan Data

app.js memanggil GET /mahasiswa dengan await fetch(), membaca body dengan .json(), lalu mengisi tabel. Dibanding contoh modul, baris tabel dibuat dengan createElement dan textContent, bukan innerHTML, sehingga isi data yang mengandung tag HTML tidak dieksekusi sebagai kode. Pemeriksaan response.ok dan hasil.success memastikan response kesalahan dari API (misalnya 500) ditangani sebagai galat. Kode di bawah juga memuat dua tugas mandiri: pesan "Memuat data..." dan "Belum ada data".

```
// Alamat dasar API (folder public/ pada XAMPP). Ubah bila nama folder proyek
berbeda.
const BASE_URL = 'http://localhost/setoran_BKPM-1/si-akademik_acara24/public';

const tbody = document.querySelector('#tabel-mahasiswa tbody');
const statusEl = document.querySelector('#status');

// Menampilkan atau menyembunyikan pesan status di bawah tabel
function tampilkanStatus(teks, jenis = '') {
    statusEl.textContent = teks;
    statusEl.className = 'status ' + jenis;
    statusEl.hidden = teks === '';
}

// Baris dibuat dengan textContent agar isi data tidak diinterpretasikan
sebagai HTML
function buatBaris(mhs) {
    const tr = document.createElement('tr');
    [mhs.nim, mhs.nama, mhs.prodi].forEach((nilai) => {
        const td = document.createElement('td');
        td.textContent = nilai;
        tr.appendChild(td);
```

```
    });
    return tr;
}

async function muatDataMahasiswa() {
    tbody.replaceChildren();
    tampilkanStatus('Memuat data...', 'info');            // Tugas mandiri 1:
indikator memuat
    try {
        const response = await fetch(`${BASE_URL}/mahasiswa`);
        const hasil = await response.json();
        if (!response.ok || !hasil.success) {
            throw new Error(hasil.message || `HTTP ${response.status}`);
        }
        if (hasil.data.length === 0) {                    // Tugas mandiri 2:
data kosong
            tampilkanStatus('Belum ada data', 'kosong');
            return;
        }
        hasil.data.forEach((mhs) => tbody.appendChild(buatBaris(mhs)));
        tampilkanStatus('');
    } catch (error) {
        console.error('Gagal memuat data:', error);
        tampilkanStatus('Gagal memuat data. Pastikan server berjalan.',
'galat');
    }
}

document.addEventListener('DOMContentLoaded', muatDataMahasiswa);
document.querySelector('#tombol-muat').addEventListener('click',
muatDataMahasiswa);
```

e. Langkah 4 - Membuat View index.html

index.html memuat tabel dengan id tabel-mahasiswa yang diisi oleh app.js, sebuah elemen status untuk pesan, dan tombol Muat ulang. Gaya tampilan ditulis pada elemen style di bagian head (tidak ditampilkan di bawah). Berikut bagian body.

```
<body>
  <main>
    <h1>Data Mahasiswa</h1>
    <p class="sub">Data diambil dari API <code>GET /mahasiswa</code>
menggunakan Fetch API.</p>
    <section class="card">
      <div class="bar">
        <strong>Daftar Mahasiswa</strong>
        <button type="button" id="tombol-muat">Muat ulang</button>
      </div>
```

```
      <table id="tabel-mahasiswa">
        <thead><tr><th>NIM</th><th>Nama</th><th>Program
Studi</th></tr></thead>
        <tbody></tbody>
      </table>
      <div id="status" class="status" role="status" aria-live="polite"
hidden></div>
    </section>
  </main>
  <script src="app.js"></script>
</body>
</html>
```

f. Langkah 5 - Pengujian Tampilan View

1. Pastikan Apache dan MySQL berjalan, dan tabel mahasiswa berisi data.

2. Buka http://localhost/setoran_BKPM-1/si-akademik_acara24/public/views/index.html pada browser (Gambar 2).

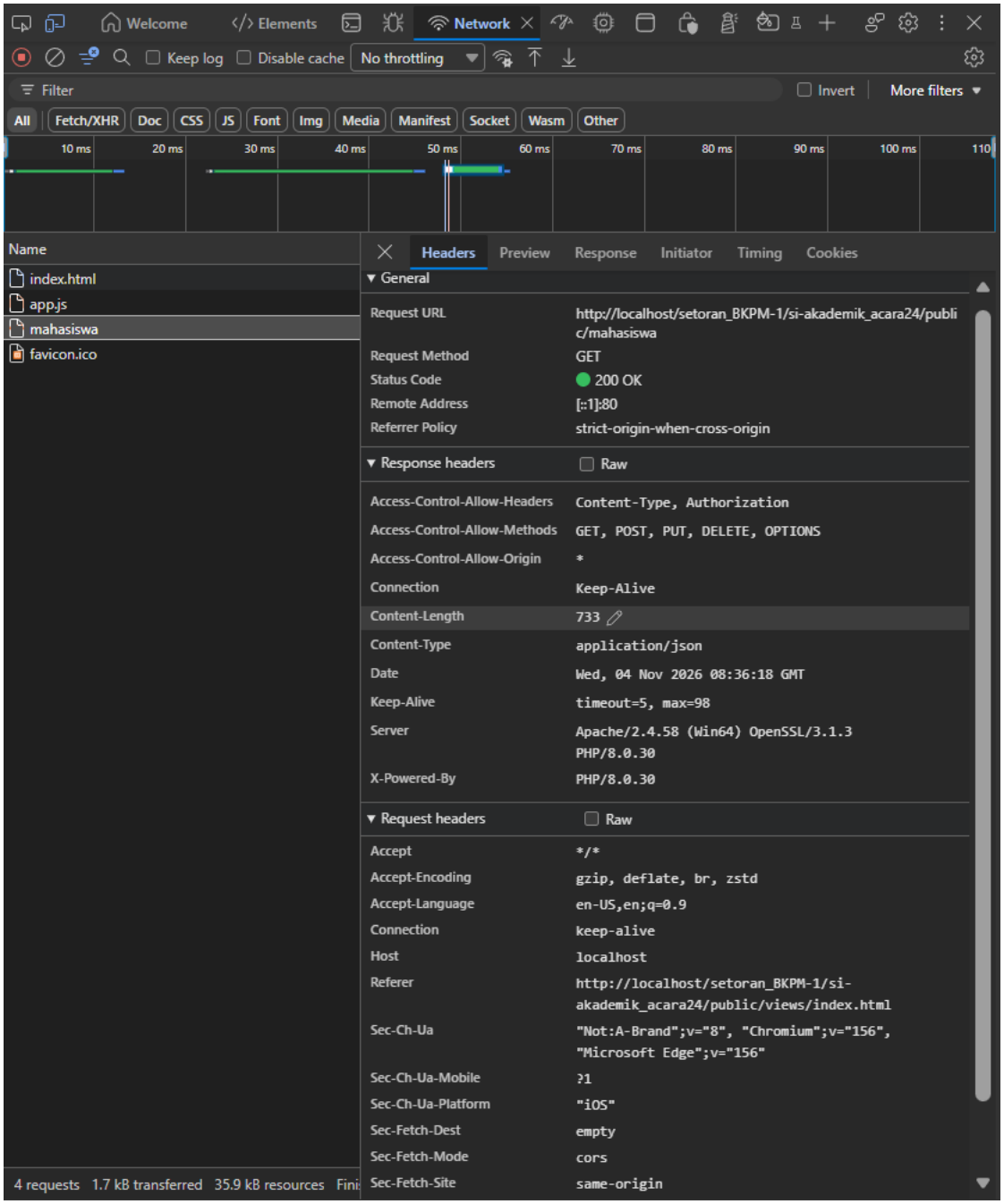
3. Bandingkan data pada tabel dengan hasil MahasiswaController::index() yang dapat dilihat lewat GET /mahasiswa.

4. Buka Developer Tools (F12). Pada tab Network, request GET /mahasiswa berstatus 200 dan response header memuat Access-Control-Allow-Origin: * (Gambar 3).

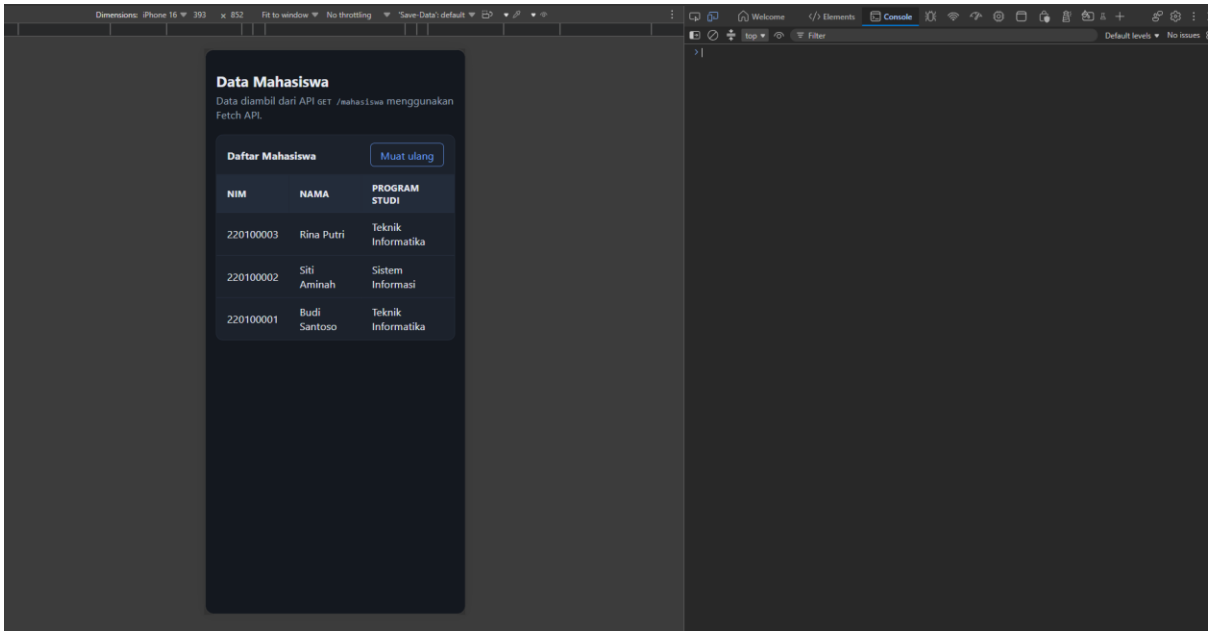
5. Pada tab Console pastikan tidak ada error CORS (Gambar 4).



Gambar 2. Tampilan halaman data mahasiswa



Gambar 3. Request GET /mahasiswa dan header CORS pada tab Network

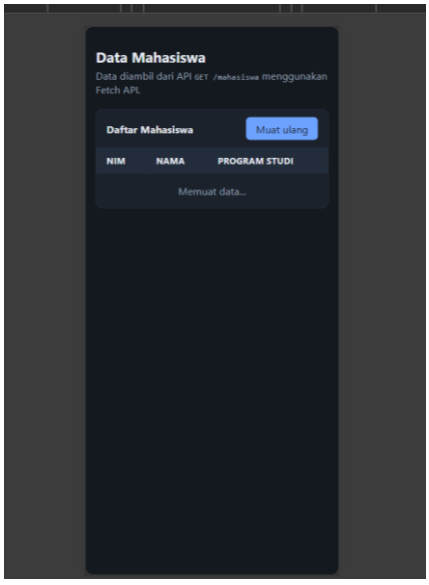


Gambar 4. Console tanpa error CORS

2. Tugas Praktik Mandiri

a. Indikator "Memuat data..."

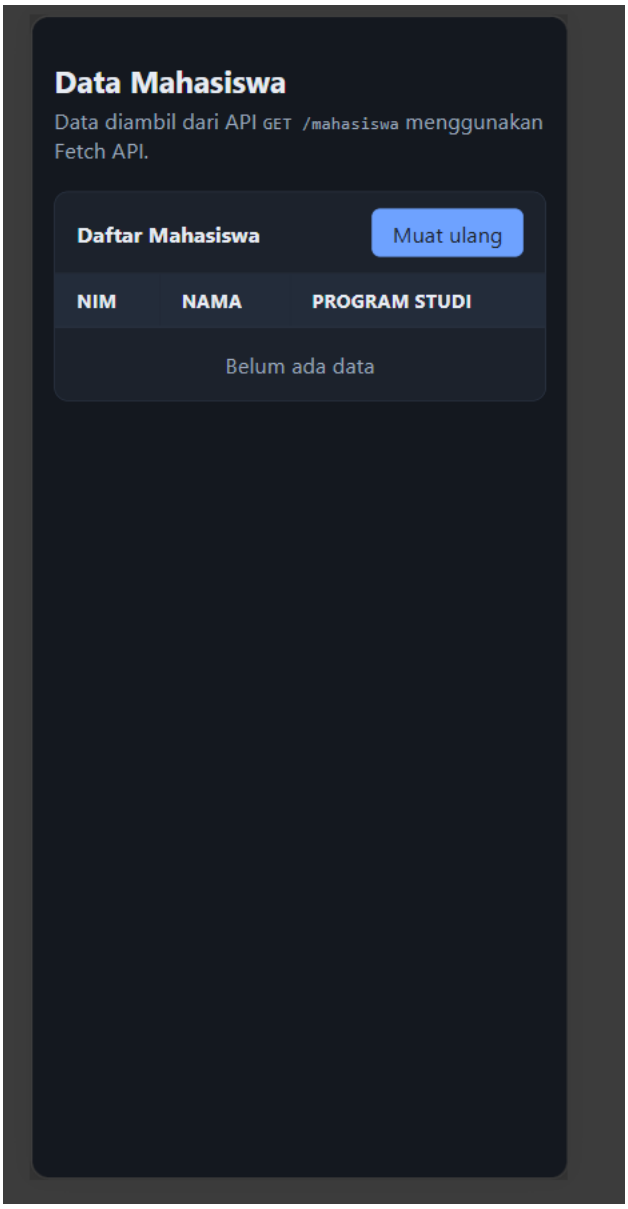
Sebelum fetch dipanggil, fungsi tampilkanStatus("Memuat data...") menampilkan pesan pada elemen status. Pesan hilang atau diganti setelah data diterima. Karena respons lokal sangat cepat, indikator hanya terlihat sekilas; untuk menampilkannya pada tangkapan layar, kecepatan jaringan dibatasi lewat DevTools (Network, throttling 3G) lalu tombol Muat ulang ditekan (Gambar 5).



Gambar 5. Indikator Memuat data...

b. Penanganan data kosong

Jika hasil.data.length bernilai 0, tabel dibiarkan kosong dan pesan "Belum ada data" ditampilkan. Kondisi ini diuji dengan mengosongkan tabel mahasiswa pada phpMyAdmin lalu memuat ulang halaman (Gambar 6).



Gambar 6. Pesan Belum ada data

3. Pengujian

Selain pengujian pada browser (Gambar 2 sampai 6), logika halaman diverifikasi dengan pengujian otomatis berbasis lingkungan DOM jsdom yang memanggil server nyata, dan header CORS diperiksa dengan curl. Seluruh pemeriksaan sesuai harapan (Tabel 3).

Tabel 3. Hasil pengujian

No	Skenario	Hasil yang diharapkan	Hasil
1	Tabel berisi 3 data mahasiswa	Tiga baris tampil, kolom NIM, Nama, Program Studi	Sesuai
2	Tabel kosong	Pesan "Belum ada data", tanpa baris	Sesuai

No	Skenario	Hasil yang diharapkan	Hasil
3	Fetch ditunda (jaringan lambat)	Pesan "Memuat data..." tampil lebih dulu, lalu diganti	Sesuai
4	Server tidak terjangkau	Pesan "Gagal memuat data", galat dicatat di console	Sesuai
5	Nama berisi tag HTML (contoh <img onerror=...>)	Tampil sebagai teks, tidak dieksekusi	Sesuai
6	Tombol Muat ulang ditekan	Data dimuat ulang tanpa menggandakan baris	Sesuai
7	GET /mahasiswa dengan header Origin asing	Response memuat Access-Control-Allow-Origin: *	Sesuai
8	Preflight OPTIONS ke /mahasiswa	204 tanpa body, header CORS lengkap	Sesuai
9	Berkas statis /views/index.html dan /views/app.js	Dilayani langsung (200), tidak lewat Router	Sesuai
10	Regresi 25 test case Acara 23	25 Pass, 0 Fail	Sesuai

4. Pembahasan

Header CORS ditulis pada Front Controller sehingga berlaku seragam untuk semua resource, bahkan untuk response kesalahan. Perlu dipahami kapan CORS benar-benar berperan. Halaman pada acara ini dibuka dari http://localhost/..., origin yang sama dengan API, sehingga browser tidak menerapkan pembatasan lintas origin dan fetch berhasil dengan atau tanpa header. Header itu baru menentukan ketika halaman dibuka dari origin lain, misalnya Live Server pada port 5500 atau berkas dibuka langsung dari disk. Nilai * cocok untuk pengembangan dan API baca-saja; pada produksi, Access-Control-Allow-Origin sebaiknya dibatasi ke origin frontend yang dikenal, terutama setelah halaman memakai autentikasi.

Penanganan OPTIONS ditambahkan di luar modul karena Router akan menjawab 405 terhadap method itu, padahal browser mengirim preflight untuk permintaan POST atau PUT dengan header Authorization seperti yang akan dipakai pada acara berikutnya. Pada sisi tampilan, penggunaan textContent menggantikan innerHTML yang dipakai pada contoh modul: dengan innerHTML, nama mahasiswa berisi tag HTML dapat dieksekusi sebagai kode (cross-site scripting), sedangkan textContent menampilkannya sebagai teks biasa. Pengujian nomor 5 pada Tabel 3 memeriksa hal ini.

Keterbatasan: halaman baru memakai GET sehingga belum mencakup login dan operasi tulis, BASE_URL masih ditulis tetap di app.js dan perlu diubah bila nama folder proyek berbeda, dan data ditampilkan sesuai urutan dari API (data terbaru lebih dulu). Pengujian jsdom tidak menggantikan browser sungguhan, khususnya penegakan CORS oleh browser, karena itu tampilan dan Developer Tools tetap diperiksa langsung pada Gambar 2 sampai 6.

KESIMPULAN

1. Header CORS berhasil ditambahkan pada Front Controller dan berlaku untuk seluruh resource, ditambah jawaban 204 untuk preflight OPTIONS.

2. View pertama pada public/views berhasil menampilkan data mahasiswa dari API menggunakan Fetch API dengan async/await dan try-catch.

3. Pengujian menunjukkan data pada tabel sesuai dengan GET /mahasiswa, tidak ada error CORS, dan 25 test case Acara 23 tetap Pass.

4. Tugas mandiri selesai: indikator "Memuat data..." dan pesan "Belum ada data" ditambahkan, ditambah pesan kesalahan bila server tidak terjangkau.

DAFTAR PUSTAKA

Pratama, R. A., & Hasanah, Q. (2026). Buku Kerja Praktik Mahasiswa (BKPM) Workshop Sistem Informasi Web Server (TIF330805). Program Studi D4 Teknik Informatika PSDKU Nganjuk, Jurusan Teknologi Informasi, Politeknik Negeri Jember.